

Compositional Symbolic Execution

Compositional Symbolic Execution in a nutshell

- Underpins tools based on ideas from separation logic: e.g. Verifast, Viper (one backend), Gillian, CN, Infer-Pulse
- Successful in both academia and industry
- Symbolic execution of **partial** symbolic state; partial symbolic heap manipulated using **consume** and **produce** operations
- First-order path constraints checked with SMT solvers (we use z3)
- **Compositionality**: possible to analyse a function separately from the rest of the codebase and record the results in function specifications in a separation logic



Theory and practice of CSE

We are interested in both the theory and practice of CSE.

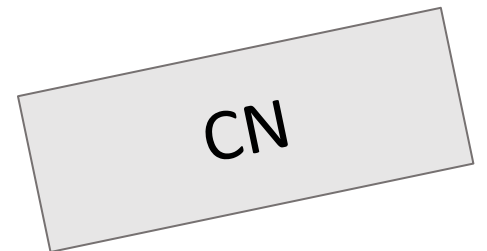
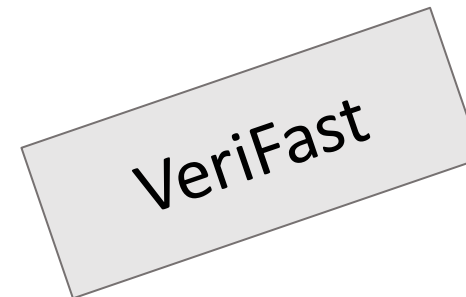
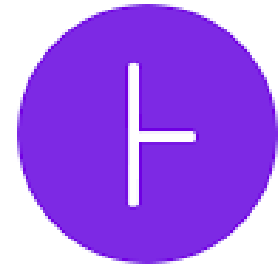
- **Theory:** we have developed the foundations of CSE theory, mechanised in Rocq, that is independent of a particular tool development.
- **Practice:** we develop and maintain the Gillian symbolic execution platform.
- Both feed into each other and inform each other!

Our CSE Theory

We differ from previous CSE theories in multiple ways.

1. **Foundational theory** independent of any tool
2. **Parametric** on the memory model
3. **Unified foundation** for compositional verification and true bug detection
4. **Standard logical definitions** used to specify functions (e.g. separation logic and incorrectness separation logic)

Gildan



CSE Components in Gillian

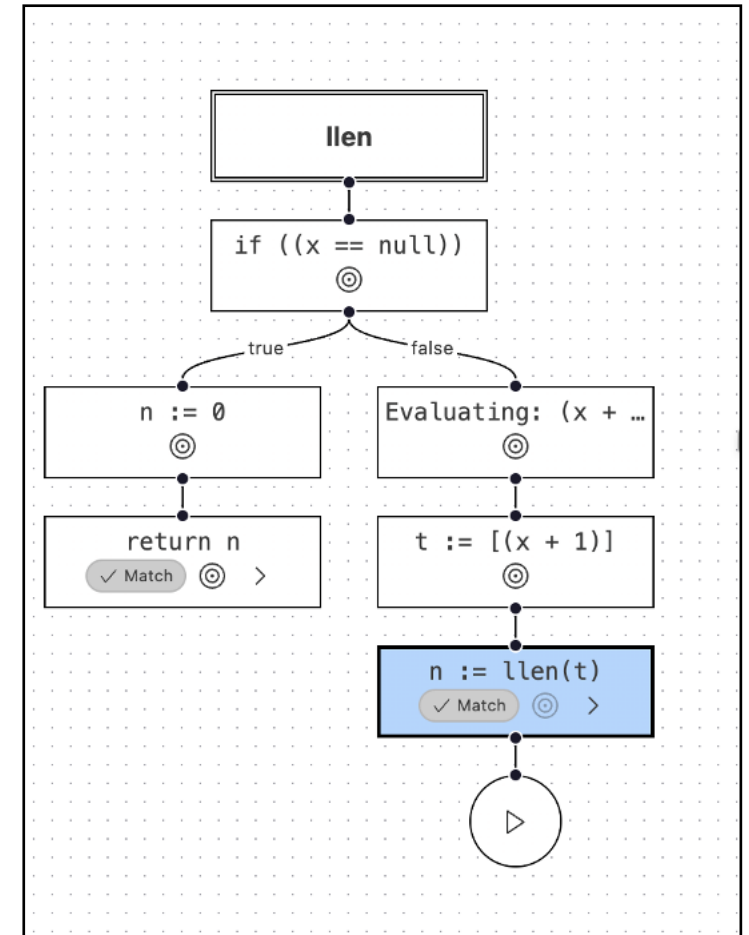
✓ **VARIABLES**

- ✓ **Program variables**
 - t = #lvar_6
 - x = #x
- ✓ **Heap**
 - ✓ #loc_0 = allocated
 - bound = 2
 - ✓ **cells**
 - 0 = #lvar_5
 - 1 = #lvar_6
- ✓ **Predicates**
 - = list(#lvar_6, #lvar_3)
- ✓ **Pure formulae**
 - = #lvar_4 == #loc_0
 - = #vs == ([#lvar_5] @ #lvar_3)
 - = #x == [#loc_0, 0]
- ✓ **Types**
 - #lvar_3 = List
 - #lvar_4 = Obj
 - #vs = List
 - #x = Pointer
- ✓ **Axioms**
 - = 0 <= len(#lvar_3)
 - = 0 <= len(#vs)
 - = 0 <= len(#x)

Symbolic State

```
13
14 predicate list(+x, vs) {
15     (x == null) * (vs == nil);
16     (x -b> #v, #z) * list(#z, #vss) * (vs == #v::#vss)
17 }
18
19 // === EDIT BELOW HERE ===
20
21 { (x == #x) * list(#x, #vs) }
22 Verify llen
23 function llen(x) {
24     if (x == null) {
25         n := 0
26     } else {
27         t := [x + 1];
28         n := llen(t);
29         n := n + 1
30     };
31     return n
32 }
33 { list(#x, #vs) * (ret == len(#vs)) }
```

Program Specification



Symbolic Execution

▾ VARIABLES
 ▾ Program variables
 t = #lvar_6
 x = #x
 ▾ Heap
 ▾ #loc_0 = allocated
 bound = 2
 ▾ cells
 0 = #lvar_5
 1 = #lvar_6

Symbolic state:
 $\hat{\sigma} = (\hat{s}, \hat{\mu}, \hat{\wp}, \hat{\pi})$

= #lvar_4 == #loc_0
 = #vs == ([#lvar_5] @ #lvar_3)
 = #x == [#loc_0, 0]

▾ Types
 #lvar_3 = List
 #lvar_4 = Obj
 #vs = List
 #x = Pointer

▾ Axioms
 = 0 <= len(#lvar_3)
 = 0 <= len(#vs)
 = 0 <= len(#x)

VARIABLES

Program variables

t = #lvar_6
 x = #x

Heap

#loc_0 = allocated

bound = 2

cells

0 = #lvar_5
 1 = #lvar_6

Predicates

= list(#lvar_6, #lvar_3)

Pure formulae

= #lvar_4 == #loc_0
 = #vs == ([#lvar_5] @ #lvar_3)
 = #x == [#loc_0, 0]

Types

#lvar_3 = List
 #lvar_4 = Obj
 #vs = List
 #x = Pointer

Axioms

= 0 <= len(#lvar_3)
 = 0 <= len(#vs)
 = 0 <= len(#x)

Symbolic store:

$\hat{s} : PVar \rightarrow_{fin} LExp$

Symbolic linear heap:

$\hat{\mu} : LExp \rightarrow_{fin} LExp$

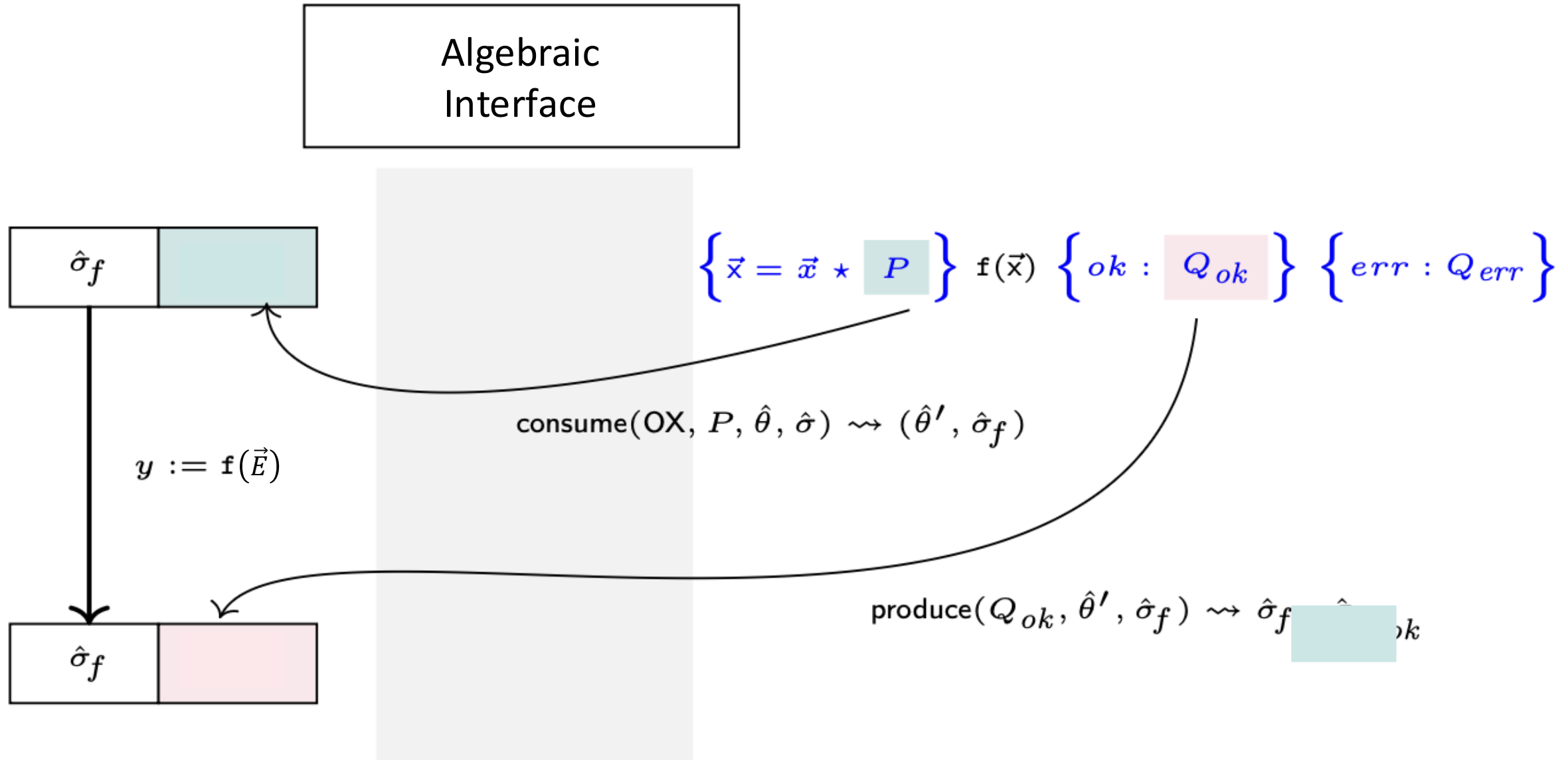
User-defined predicates:

$\hat{\wp} : \text{type omitted}$

Symbolic path condition:

$\hat{\pi} : LExp$

Compositional Symbolic Execution: Function Compositionality



CSE for *LLen* – Recursive case

Approach: to verify this function specification using compositional symbolic execution

```
{x = x * list(x, α)}  
LLen(x) {  
  if(x = null){  
    n := 0;  
  } else {  
    t := [x + 1];  
    n := LLen(t);  
    n := n + 1;  
  }  
  return n;  
}  
{ok: list(x, α) * ret = |α|}
```

Produce (i.e. assume) initial symbolic state from assertion to start symbolic execution

For function call, **consume** precondition and then **produce** postcondition

Consume (i.e. assert) final symbolic state and check it matches specification in the assertion

Produce the initial symbolic state given $list(x, \alpha)$ in the precondition

Initial symbolic state also initialises parameter variable (x) and variables in the function body (n, t)

$\{x = x * list(x, \alpha)\}$

$LLen(x)\{$

// $\widehat{\sigma}_1 = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto null], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], true) : produce$

$if(x = null)\{$

// $\widehat{\sigma}_2 = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto null], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], \hat{x} = null) : if branch$

// $\widehat{\sigma}_3 = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto null], \widehat{\mu}_\emptyset, \emptyset, \hat{x} = null \wedge \hat{\alpha} = []) : unfolding list predicate$

$n := 0;$

// $\widehat{\sigma}_4 = ([x \mapsto \hat{x}, n \mapsto 0, t \mapsto null], \widehat{\mu}_\emptyset, \emptyset, \hat{x} = null \wedge \hat{\alpha} = []) : assignment$

// $\widehat{\sigma}_5 = ([x \mapsto \hat{x}, n \mapsto 0, t \mapsto null], \widehat{\mu}_\emptyset, \emptyset, \hat{x} = null \wedge \hat{\alpha} = []) : folding list predicate$

$\} else \{$

$\dots$

Analogous to intermediate assertions in proof sketches

$\{x = x * n, t = null * list(x, \alpha) * x = null\}$

$\{x = x * list(x, \alpha)\}$

$LLen(x)\{$

$// \widehat{\sigma}_1 = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto null], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], true) : produce$

$if(x = null)\{$

$// \widehat{\sigma}_2 = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto null], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], \hat{x} = null) : if branch$

$// \widehat{\sigma}_3 = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto null], \widehat{\mu}_\emptyset, \emptyset, \hat{x} = null \wedge \hat{\alpha} = []) : unfolding list predicate$

$n := 0;$

$// \widehat{\sigma}_4 = ([x \mapsto \hat{x}, n \mapsto 0, t \mapsto null], \widehat{\mu}_\emptyset, \emptyset, \hat{x} = null \wedge \hat{\alpha} = []) : assignment$

$// \widehat{\sigma}_5 = ([x \mapsto \hat{x}, n \mapsto 0, t \mapsto null], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], \hat{x} = null \wedge \hat{\alpha} = []) : folding list predicate$

$\} else \{$

$\dots$

} else {

// $\widehat{\sigma}_6 = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto null], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], \hat{x} \neq null) : \text{else branch}$

// $\widehat{\sigma}_7 = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto null], [\hat{x} \mapsto \hat{v}, \hat{x} + 1 \mapsto \hat{y}], [list(\hat{y}, \hat{\beta})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta}) : \text{unfold list}$

$t := [x + 1];$

// $\widehat{\sigma}_8 = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto \hat{y}], [\hat{x} \mapsto \hat{v}, \hat{x} + 1 \mapsto \hat{y}], [list(\hat{y}, \hat{\beta})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta}) : \text{lookup}$

$n := LLen(t);$

// $\widehat{\sigma}_9 = ([x \mapsto \hat{x}, n \mapsto |\hat{\beta}|, t \mapsto \hat{y}], [\hat{x} \mapsto \hat{v}, \hat{x} + 1 \mapsto \hat{y}], [list(\hat{y}, \hat{\beta})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta}) : \text{fcall}$

$n := n + 1;$

// $\widehat{\sigma}_{10} = ([x \mapsto \hat{x}, n \mapsto |\hat{\beta}| + 1, t \mapsto \hat{y}], [\hat{x} \mapsto \hat{v}, \hat{x} + 1 \mapsto \hat{y}], [list(\hat{y}, \hat{\beta})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta}) : \text{assign}$

// $\widehat{\sigma}_{11} = ([x \mapsto \hat{x}, n \mapsto |\hat{\beta}| + 1, t \mapsto \hat{y}], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta}) : \text{fold list}$

}

return n ;

// note: we now have two branches, so we must consider changes in both symbolic states

/ $\widehat{\sigma}_{5.1} = ([x \mapsto \hat{x}, n \mapsto 0, t \mapsto null, ret \mapsto 0], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], \hat{x} = null \wedge \hat{\alpha} = [])$*

*$\widehat{\sigma}_{11.1} = ([x \mapsto \hat{x}, n \mapsto |\hat{\beta}| + 1, t \mapsto \hat{y}, ret \mapsto |\hat{\beta}| + 1], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta})$ */*

}

{ok: $list(x, \alpha) * ret = |\alpha|$ }

Consume part of $\widehat{\sigma}_8$ that matches the precondition when LLen is given t as input

} else {

Results in the following intermediate symbolic state:

$$\widehat{\sigma}' = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto \hat{y}], [\hat{x} \mapsto \hat{v}, \hat{x} + 1 \mapsto \hat{y}], \emptyset, \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta})$$

//

$t := [x + 1];$

// $\widehat{\sigma}_8 = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto \hat{y}], [\hat{x} \mapsto \hat{v}, \hat{x} + 1 \mapsto \hat{y}], [list(\hat{y}, \hat{\beta})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta})$

$n := LLen(t);$

// $\widehat{\sigma}_9 = ([x \mapsto \hat{x}, n \mapsto |\hat{\beta}|, t \mapsto \hat{y}], [\hat{x} \mapsto \hat{v}, \hat{x} + 1 \mapsto \hat{y}], [list(\hat{y}, \hat{\beta})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta})$

$n := n + 1;$

// $\widehat{\sigma} \mapsto |\hat{\beta}| + 1, t \mapsto \hat{v}, [\hat{x} \mapsto \hat{v}, \hat{x} + 1 \mapsto \hat{v}], [list(\hat{v}, \hat{\beta})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta}) : assign$

Produce postcondition given $\widehat{\sigma}'$ (**ret** maps to n), which yields $\widehat{\sigma}_9$

// note: we now have two branches, so we must consider changes in both symbolic states

/* $\widehat{\sigma}_{5.1} = ([x \mapsto \hat{x}, n \mapsto 0, t \mapsto null, ret \mapsto 0], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], \hat{x} = null \wedge \hat{\alpha} = [])$

$\widehat{\sigma}_{11.1} = ([x \mapsto \hat{x}, n \mapsto |\hat{\beta}| + 1, t \mapsto \hat{y}, ret \mapsto |\hat{\beta}| + 1], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta})$

}

{ok: $list(x, \alpha) * ret = |\alpha|$ }

Recall:

$\{x = x * list(x, \alpha)\}$

$LLen(x) \{ \dots \}$

$\{list(x, \alpha) * ret = |\alpha|\}$

} else {

// $\widehat{\sigma}_6 = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto null], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], \hat{x} \neq null) : \text{else branch}$

// $\widehat{\sigma}_7 = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto null], [\hat{x} \mapsto \hat{v}, \hat{x} + 1 \mapsto \hat{y}], [list(\hat{y}, \hat{\beta})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta}) : \text{unfold list}$

$t := [x + 1];$

// $\widehat{\sigma}_8 = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto \hat{y}], [\hat{x} \mapsto \hat{v}, \hat{x} + 1 \mapsto \hat{y}], [list(\hat{y}, \hat{\beta})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta}) : \text{lookup}$

$n := LLen(t);$

// $\widehat{\sigma}_9 = ([x \mapsto \hat{x}, n \mapsto |\hat{\beta}|, t \mapsto \hat{y}], [\hat{x} \mapsto \hat{v}, \hat{x} + 1 \mapsto \hat{y}], [list(\hat{y}, \hat{\beta})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta}) : \text{fcall}$

$n := n + 1;$

// $\widehat{\sigma}_{10} = ([x \mapsto \hat{x}, n \mapsto |\hat{\beta}| + 1, t \mapsto \hat{y}], [\hat{x} \mapsto \hat{v}, \hat{x} + 1 \mapsto \hat{y}], [list(\hat{y}, \hat{\beta})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta}) : \text{assign}$

// $\widehat{\sigma}_{11} = ([x \mapsto \hat{x}, n \mapsto |\hat{\beta}| + 1, t \mapsto \hat{y}], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta}) : \text{fold list}$

}

return n ;

// note: we now have two branches, so we must consider changes in both symbolic states

/* $\widehat{\sigma}_{5.1} = ([x \mapsto \hat{x}, n \mapsto 0, t \mapsto null, ret \mapsto 0], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], \hat{x} = null \wedge \hat{\alpha} = [])$

$\widehat{\sigma}_{11.1} = ([x \mapsto \hat{x}, n \mapsto |\hat{\beta}| + 1, t \mapsto \hat{y}, ret \mapsto |\hat{\beta}| + 1], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta}) */$

}

{ok: $list(x, \alpha) * ret = |\alpha|$ }

} else {

// $\widehat{\sigma}_6 = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto null], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], \hat{x} \neq null) : \text{else branch}$

// $\widehat{\sigma}_7 = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto null], [\hat{x} \mapsto \hat{v}, \hat{x} + 1 \mapsto \hat{y}], [list(\hat{y}, \hat{\beta})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta}) : \text{unfold list}$

$t := [x + 1];$

// $\widehat{\sigma}_8 = ([x \mapsto \hat{x}, n \mapsto null, t \mapsto \hat{y}], [\hat{x} \mapsto \hat{v}, \hat{x} + 1 \mapsto \hat{y}], [list(\hat{y}, \hat{\beta})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta}) : \text{lookup}$

$n := LLen(t);$

// $\widehat{\sigma}_9 = ([x \mapsto \hat{x}, n \mapsto |\hat{\beta}|, t \mapsto \hat{y}], [\hat{x} \mapsto \hat{v}, \hat{x} + 1 \mapsto \hat{y}], [list(\hat{y}, \hat{\beta})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta}) : \text{fcall}$

r

Consume postcondition from $\widehat{\sigma}_{5.1}$ and check that the result satisfies the empty assertion
(i.e. $heap = \widehat{\mu}_\emptyset$ and $pred = \emptyset$)

sign

}

return n ;

/* $\widehat{\sigma}_{5.1} = ([x \mapsto \hat{x}, n \mapsto 0, t \mapsto null, \mathbf{ret} \mapsto 0], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], \hat{x} = null \wedge \hat{\alpha} = [])$

$\widehat{\sigma}_{11.1} = ([x \mapsto \hat{x}, n \mapsto |\hat{\beta}| + 1, t \mapsto \hat{y}, \mathbf{ret} \mapsto |\hat{\beta}| + 1], \widehat{\mu}_\emptyset, [list(\hat{x}, \hat{\alpha})], \hat{x} \neq null \wedge \hat{\alpha} = \hat{v} :: \hat{\beta})$ */

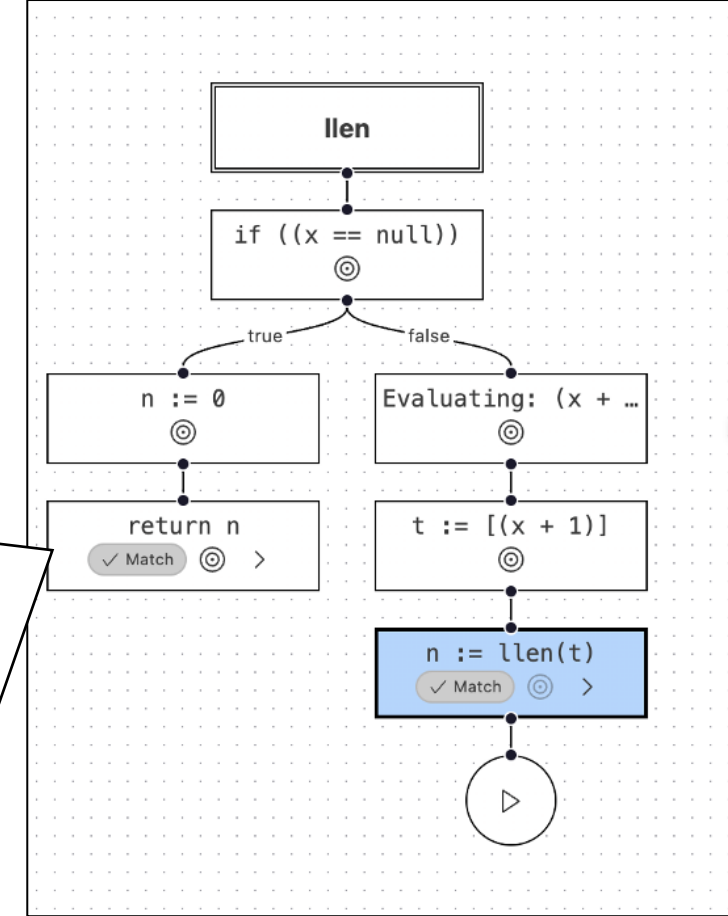
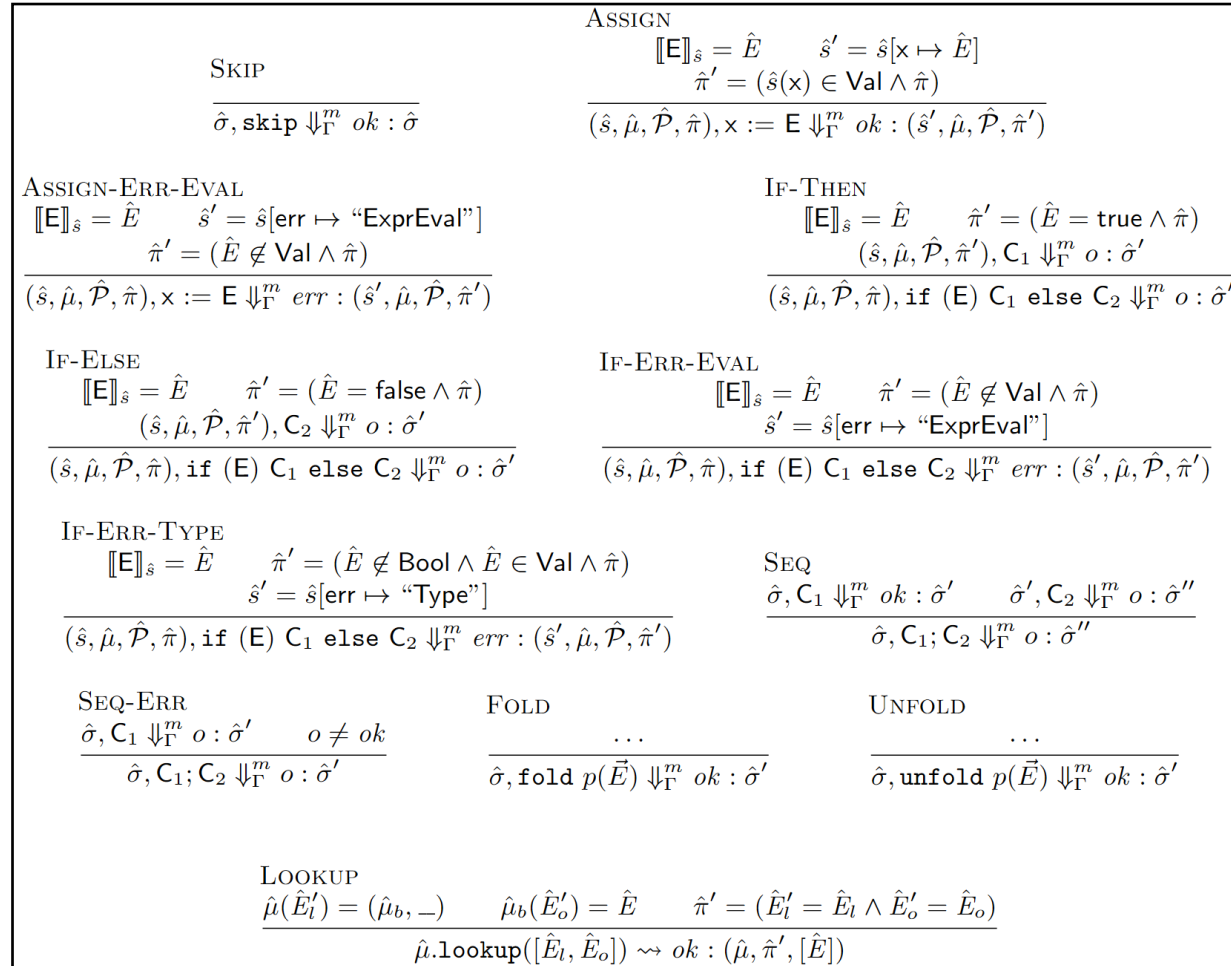
}

{ok: $list(x, \alpha) * \mathbf{ret} = |\alpha|$ }

Consume postcondition from $\widehat{\sigma}_{11.1}$ and check that the result satisfies the empty assertion
(i.e. $heap = \widehat{\mu}_\emptyset$ and $pred = \emptyset$)

CSE Parametric on Memory Model

Compositional Symbolic Execution



Memory Models in Program Reasoning

Memory model = Memory structure
+
Read and write actions

Memory Models in Program Reasoning

Requirement : Different applications require different memory models

- 1. Different memory models for different languages

E.g.: C = bytes, raw pointers, etc.

E.g.: JavaScript = objects, prototype chains, etc.

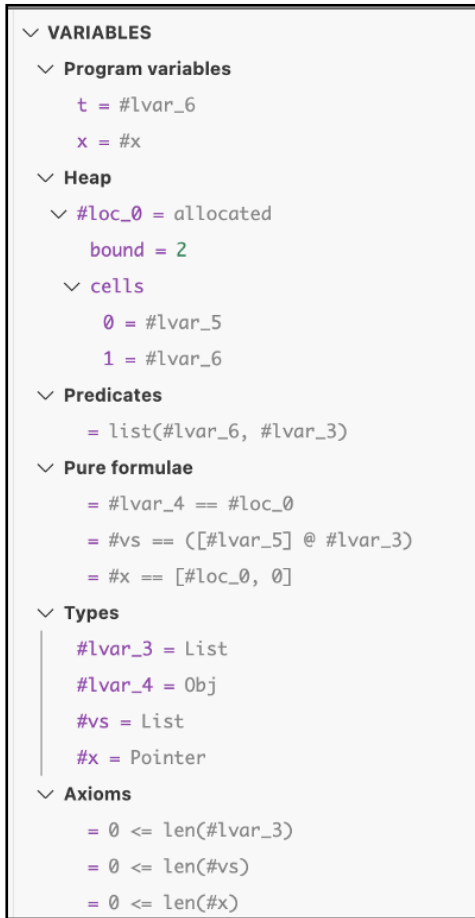
- 2. Different types of analyses require different types of ghost state

E.g.: exclusive ownership vs. fractional ownership

- 3. Other different requirements

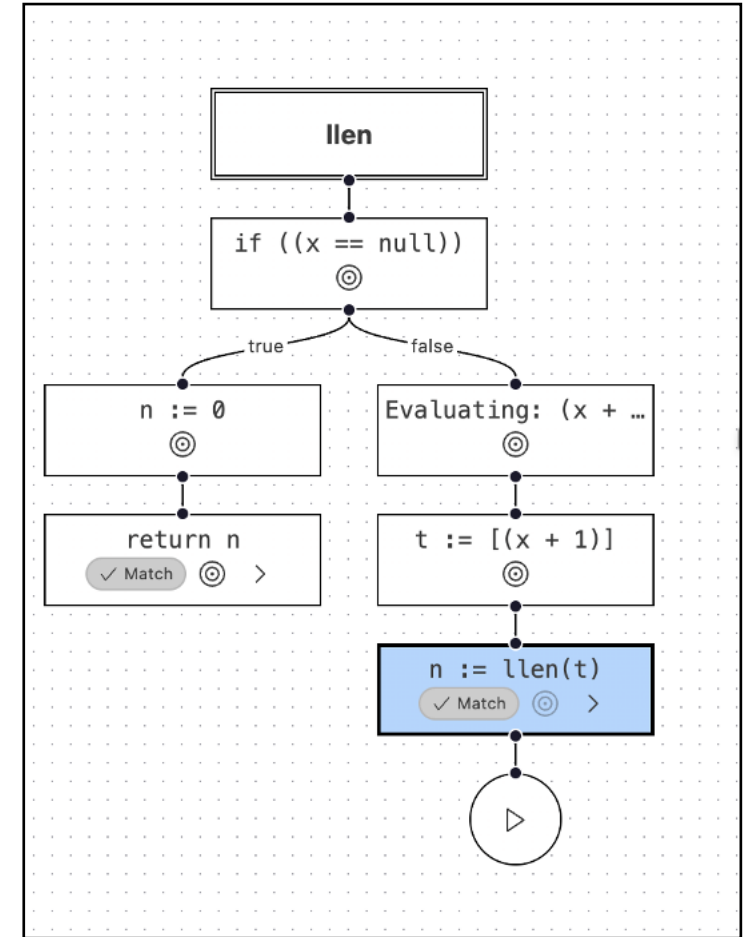
E.g.: performance, implementation effort, automation support, etc.

Gillian-While (WiSL)



```
13
14 predicate list(+x, vs) {
15     (x == null) * (vs == nil);
16     (x -b> #v, #z) * list(#z, #vss) * (vs == #v::#vss)
17 }
18
19 // === EDIT BELOW HERE ===
20
21 { (x == #x) * list(#x, #vs) }
22 Verify llen
23 function llen(x) {
24     if (x == null) {
25         n := 0
26     } else {
27         t := [x + 1];
28         n := llen(t);
29         n := n + 1
30     };
31     return n
32 }
33 { list(#x, #vs) * (ret == len(#vs)) }
```

Program Specification



Symbolic Execution

▾ VARIABLES
 ▾ Program variables
 t = #lvar_6
 x = #x
 ▾ Heap
 ▾ #loc_0 = allocated
 bound = 2
 ▾ cells
 0 = #lvar_5
 1 = #lvar_6

Symbolic state:
 $\hat{\sigma} = (\hat{s}, \hat{\mu}, \hat{\wp}, \hat{\pi})$

= #lvar_4 == #loc_0
 = #vs == ([#lvar_5] @ #lvar_3)
 = #x == [#loc_0, 0]

▾ Types
 #lvar_3 = List
 #lvar_4 = Obj
 #vs = List
 #x = Pointer

▾ Axioms
 = 0 <= len(#lvar_3)
 = 0 <= len(#vs)
 = 0 <= len(#x)

VARIABLES

Program variables

t = #lvar_6
 x = #x

Heap

#loc_0 = allocated

bound = 2

cells

0 = #lvar_5
 1 = #lvar_6

Predicates

= list(#lvar_6, #lvar_3)

Pure formulae

= #lvar_4 == #loc_0
 = #vs == ([#lvar_5] @ #lvar_3)
 = #x == [#loc_0, 0]

Types

#lvar_3 = List
 #lvar_4 = Obj
 #vs = List
 #x = Pointer

Axioms

= 0 <= len(#lvar_3)
 = 0 <= len(#vs)
 = 0 <= len(#x)

Symbolic store:

$\hat{s} : PVar \rightarrow_{fin} LExp$

Symbolic WiSL memory:

$\hat{\mu} : LExp \rightarrow_{fin} ((LExp \rightarrow_{fin} LExp) \times LExp^?)_{\emptyset}$

User-defined predicates:

$\hat{\wp} : \text{type omitted}$

Symbolic path condition:

$\hat{\pi} : LExp$

Example CSE Instantiation: WiSL

- Concrete partial memory model:

$$\mu : Loc \rightarrow_{fin} ((Nat \rightarrow_{fin} Val) \times Nat^?)_{\emptyset}$$

- Well-formedness condition
(e.g. each cell's offset is less than the block's bound)
- Resource assertions: $E \mapsto E \mid bound(E, E) \mid E \mapsto \emptyset$
- Symbolic partial memory model:

$$\hat{\mu} : LExp \rightarrow_{fin} ((LExp \rightarrow_{fin} LExp) \times LExp^?)_{\emptyset}$$

- $consume_{Res}$ and $produce_{Res}$
(consuming & producing resource assertions)

Memory Soundness

$\theta : LVar \rightarrow_{fin} LVal$ is the logical environment

$$\boxed{\theta}, (s, \mu) \models (\hat{s}, \hat{\mu}, \hat{\mathcal{P}}, \hat{\pi}) \stackrel{\text{def}}{=} \exists \mu_1, \mu_2.$$

$$\boxed{\mu = \mu_1 \cdot \mu_2}$$

$$\text{and } \theta, s \models_{\text{Sto}} \hat{s}$$

$$\text{and } \boxed{\theta, \mu_1 \models_{\text{Mem}} \hat{\mu}}$$

$$\text{and } \boxed{\theta, \mu_2 \models_{\text{Pred}} \hat{\mathcal{P}}}$$

$$\text{and } \llbracket \hat{\pi} \rrbracket_{\theta} = \text{true}$$

In the general definition of partial memory, memory composition is a requirement of the instance.

Similarly, the satisfaction relations for memory and predicates are a requirement of the instance.

WISL actions

WISL Actions

WISL memory actions — lookup, mutate, allocation and deallocation — are particular instances of general CSE actions.

$$\frac{\text{LOOKUP} \quad \hat{\mu}(\hat{E}'_l) = (\hat{\mu}_b, -) \quad \hat{\mu}_b(\hat{E}'_o) = \hat{E} \quad \hat{\pi}' = (\hat{E}'_l = \hat{E}_l \wedge \hat{E}'_o = \hat{E}_o)}{\hat{\mu}.\text{lookup}([\hat{E}_l, \hat{E}_o]) \rightsquigarrow ok : (\hat{\mu}, \hat{\pi}', [\hat{E}])}$$

Frame Properties for Memory Actions

OX frame

“If memory is removed by an action, (non-missing) behaviour is unaffected”

If $(\mu \cdot \mu_f). \alpha(\vec{v}) \rightsquigarrow_{\mathcal{M}} o : (\mu', \vec{v}')$
then $\exists \mu'', \vec{v}'', o'. \mu. \alpha(\vec{v}) \rightsquigarrow_{\mathcal{M}} o' : (\mu'', \vec{v}'')$ and
 $(o' \neq \text{miss} \Rightarrow (o' = o \text{ and } \vec{v}'' = \vec{v}' \text{ and } \mu' = \mu'' \cdot \mu_f))$

UX frame

“If memory is added by an action, (non-missing) behaviour is unaffected”

If $\mu. \alpha(\vec{v}) \rightsquigarrow_{\mathcal{M}} o : (\mu', \vec{v}')$ and $o \neq \text{miss}$ and $\mu' \cdot \mu_f$ is defined
then $(\mu \cdot \mu_f). \alpha(\vec{v}) \rightsquigarrow_{\mathcal{M}} o : (\mu' \cdot \mu_f, \vec{v}')$

Result

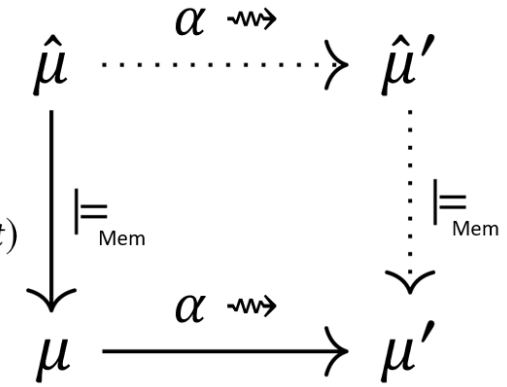
The WiSL memory model satisfies the OX and UX frame properties.

OX and UX Soundness for actions

OX soundness

All *concrete* actions can be simulated by a corresponding *symbolic* action

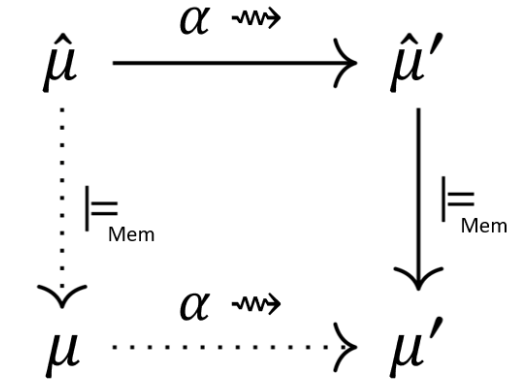
If $\mu.\alpha(\vec{v}) \rightsquigarrow o : (\mu', \vec{v}')$ and $\theta, \mu \models_{\text{Mem}} \hat{\mu}$ and $\llbracket \vec{E} \rrbracket_{\theta} = \vec{v}$ and
 $(\forall o, \hat{\mu}', \hat{\pi}', \vec{E}'. \hat{\mu}.\alpha(\vec{E}) \rightsquigarrow o : (\hat{\mu}', \hat{\pi}', \vec{E}') \text{ and } \llbracket \hat{\pi}' \rrbracket_{\theta} = \text{true and } \llbracket \vec{E}' \rrbracket_{\theta} \neq \perp \Rightarrow o \neq \text{abort})$
 then $\exists \hat{\mu}', \hat{\pi}', \vec{E}', \theta'. \hat{\mu}.\alpha(\vec{E}) \rightsquigarrow o : (\hat{\mu}', \hat{\pi}', \vec{E}')$ and $\theta' \upharpoonright_{\text{lv}(\hat{\mu})} = \theta$
 and $\llbracket \hat{\pi}' \rrbracket_{\theta'} = \text{true and } \theta', \mu' \models_{\text{Mem}} \hat{\mu}' \text{ and } \llbracket \vec{E}' \rrbracket_{\theta'} = \vec{v}'$



UX soundness

All *symbolic* actions can be simulated by a corresponding *concrete* action

If $\hat{\mu}.\alpha(\vec{E}) \rightsquigarrow o : (\hat{\mu}', \hat{\pi}', \vec{E}')$ and $o \neq \text{abort}$ and $\llbracket \vec{E} \rrbracket_{\theta} = \vec{v}$ and
 $\llbracket \hat{\pi}' \rrbracket_{\theta} = \text{true and } \theta, \mu' \models_{\text{Mem}} \hat{\mu}' \text{ and } \llbracket \vec{E}' \rrbracket_{\theta} = \vec{v}'$
 then $\exists \mu. \theta, \mu \models_{\text{Mem}} \hat{\mu} \text{ and } \mu.\alpha(\vec{v}) \rightsquigarrow o : (\mu', \vec{v}')$



Result

WiSL actions are sound.

Reasoning with function specifications

CSE

Functions with rich function specifications expressed using user-defined predicates to capture operations over e.g. pointer-based data structures

Consume and produce

- Reasoning about function calls using function specifications
- Verification of function specifications
- Folding and unfolding of user-defined predicates
- Using and proving lemmas (similar to functions)

Compositionality: Function Call Rule

$$\Gamma(f)|_m = \langle\langle \vec{x} = \vec{x} \star P \rangle\rangle f(\vec{x}) \langle\langle ok : Q_{ok} \rangle\rangle \langle\langle err : Q_{err} \rangle\rangle$$

$$\llbracket \vec{E} \rrbracket_{\hat{s}} \Downarrow \vec{E} \text{ and } \hat{\theta} = [\vec{x} \mapsto \vec{E}]$$

$$\hat{\pi}' = (\vec{E} \in \text{Val} \wedge \hat{\pi})$$

$$\text{consume}(m, P, \hat{\theta}, (\hat{s}, \hat{\mu}, \hat{\mathcal{P}}, \hat{\pi}')) \rightsquigarrow (ok, \hat{\theta}', (\hat{s}, \hat{\mu}_f, \hat{\mathcal{P}}_f, \hat{\pi}''))$$

$\hat{r}$ fresh

$$Q'_{ok} = Q_{ok}[\hat{r}/\text{ret}] \text{ and } \hat{\theta}'' = \hat{\theta}'[r \mapsto \hat{r}]$$

$$\hat{\pi}''' = (\hat{r} \in \text{Val} \wedge \hat{\pi}'')$$

$$\text{produce}(Q'_{ok}, \hat{\theta}'', (\hat{s}, \hat{\mu}_f, \hat{\mathcal{P}}_f, \hat{\pi}''')) \rightsquigarrow (\hat{s}, \hat{\mu}', \hat{\mathcal{P}}', \hat{\pi}''''')$$

$$\hat{\pi}'''' = (\hat{s}(y) \in \text{Val} \wedge \hat{\pi}''''')$$

get function specification

evaluate function parameters

successful evaluation in path condition

consume pre-condition

fresh variable for return value

set up return value

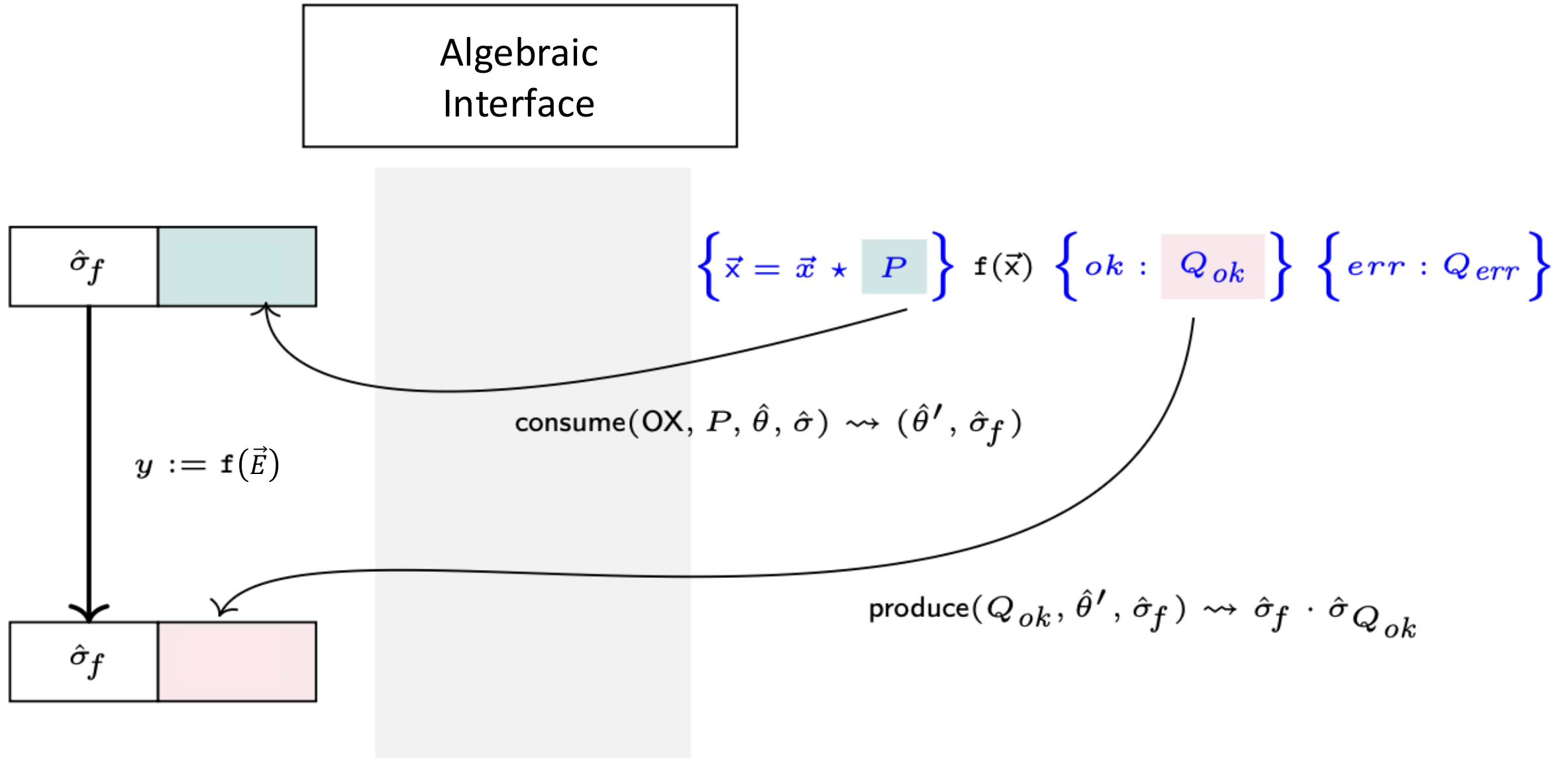
include $\hat{r}$ in path condition

produce post-condition

include y in path condition

$$(\hat{s}, \hat{\mu}, \hat{\mathcal{P}}, \hat{\pi}), y := f(\vec{E}) \Downarrow_{\Gamma}^m ok : (\hat{s}[y \mapsto \hat{r}], \hat{\mu}', \hat{\mathcal{P}}', \hat{\pi}''''')$$

Compositional Symbolic Execution: Function Compositionality



Soundness of Consume and Produce

How do we prove soundness of constructs defined in terms of consume and produce, such as function call and fold/unfold?

Our answer: algebraic interface for consume and produce

Our interface captures mathematically what has been illustrated using pictures in the previous slides

Mathematical definitions allow us to show:

- Consume-and-produce operations are sound assuming the interface holds and the memory actions satisfy frame.
- The Gillian implementation of consume and produce satisfies the interface.

Our Consume and Produce Interface

PROPERTY 1 (OX SOUNDNESS: CONSUME).

If $m \in \{OX, EX\}$ and
 $(\forall \hat{\theta}', \hat{\sigma}', o. \text{consume}(m, P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow (o, \hat{\theta}', \hat{\sigma}') \text{ and } \text{SAT}(\hat{\sigma}') \Rightarrow o \neq \text{abort})$
and $\theta, s, \mu \models \hat{\sigma}$
then $\exists \mu_P, \mu_f, \hat{\theta}', \hat{\sigma}_f. \text{consume}(m, P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow (ok, \hat{\theta}', \hat{\sigma}_f)$ and $\mu = \mu_P \cdot \mu_f$
and $\theta, s, \mu_P \models \hat{\theta}'(P)$ and $\theta, s, \mu_f \models \hat{\sigma}_f$

PROPERTY 2 (UX SOUNDNESS: CONSUME).

If $\text{consume}(m, P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow (ok, \hat{\theta}', \hat{\sigma}_f)$
and $\theta, s, \mu_P \models \hat{\theta}'(P)$ and $\theta, s, \mu_f \models \hat{\sigma}_f$ and $(\mu_P \cdot \mu_f)$ is defined
then $\theta, s, (\mu_P \cdot \mu_f) \models \hat{\sigma}$

PROPERTY 3 (OX SOUNDNESS: PRODUCE). Let $\text{lv}(P) \subseteq \text{dom}(\hat{\theta})$.

If $\theta, s, \mu_P \models \hat{\theta}'(P)$ and $\theta, s, \mu \models \hat{\sigma}$ and $(\mu_P \cdot \mu_f)$ is defined
then $\exists \theta, \hat{\sigma}'. \theta'|_{\text{lv}(\hat{\sigma})} = \theta$ and $\text{produce}(P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow \hat{\sigma}'$ and $\theta, s, (\mu_P \cdot \mu_f) \models \hat{\sigma}'$

PROPERTY 4 (UX SOUNDNESS: PRODUCE).

If $\text{produce}(P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow \hat{\sigma}'$ and $\theta, s, \mu \models \hat{\sigma}$
then $\exists \mu_P, \mu_f. \mu = \mu_P \cdot \mu_f$ and $\theta, s, \mu_P \models \hat{\theta}(P)$ and $\theta, s, \mu_f \models \hat{\sigma}$

Our Consume and Produce Interface

PROPERTY 1 (OX SOUNDNESS: CONSUME).

If $m \in \{OX, EX\}$ and
 $(\forall \hat{\theta}', \hat{\sigma}', o. \text{consume}(m, P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow (o, \hat{\theta}', \hat{\sigma}') \text{ and } \text{SAT}(\hat{\sigma}') \Rightarrow o \neq \text{abort})$
and $\theta, s, \mu \models \hat{\sigma}$
then $\exists \mu_P, \mu_f, \hat{\theta}', \hat{\sigma}_f. \text{consume}(m, P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow (ok, \hat{\theta}', \hat{\sigma}_f) \text{ and } \mu = \mu_P \cdot \mu_f$

Remark: The interface is independent of the

- memory model
- language
- implementation

PROPERTY 2

If
then $\theta, s, (\mu_P \cdot \mu_f) \models \hat{\sigma}$

PROPERTY 3 (OX SOUNDNESS: PRODUCE). *Let* $\text{lv}(P) \subseteq \text{dom}(\hat{\theta})$.

If $\theta, s, \mu_P \models \hat{\theta}'(P)$ and $\theta, s, \mu \models \hat{\sigma}$ and $(\mu_P \cdot \mu_f)$ is defined
then $\exists \theta, \hat{\sigma}'. \theta'|_{\text{lv}(\hat{\sigma})} = \theta$ and $\text{produce}(P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow \hat{\sigma}'$ and $\theta, s, (\mu_P \cdot \mu_f) \models \hat{\sigma}'$

PROPERTY 4 (UX SOUNDNESS: PRODUCE).

If $\text{produce}(P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow \hat{\sigma}'$ and $\theta, s, \mu \models \hat{\sigma}'$
then $\exists \mu_P, \mu_f. \mu = \mu_P \cdot \mu_f$ and $\theta, s, \mu_P \models \hat{\theta}(P)$ and $\theta, s, \mu_f \models \hat{\sigma}$

Our Consume and Produce Interface

PROPERTY 1 (OX SOUNDNESS: CONSUME).

If $m \in \{OX, EX\}$ and
 $(\forall \hat{\theta}', \hat{\sigma}', o. \text{consume}(m, P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow (o, \hat{\theta}', \hat{\sigma}') \text{ and } \text{SAT}(\hat{\sigma}') \Rightarrow o \neq \text{abort})$
and $\theta, s, \mu \models \hat{\sigma}$
then $\exists \mu_P, \mu_f, \hat{\theta}', \hat{\sigma}_f. \text{consume}(m, P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow (\text{ok}, \hat{\theta}', \hat{\sigma}_f) \text{ and } \mu = \mu_P \cdot \mu_f$
and $\theta, s, \mu_P \models \hat{\theta}'(P) \text{ and } \theta, s, \mu_f \models \hat{\sigma}_f$

Result: Function Call Soundness

PROPERTY 2 If the consume and produce operations satisfy these OX/UX properties, then the function call rule is OX/UX sound.

If $\theta, s, \mu_P \models \hat{\theta}'(P) \text{ and } \theta, s, \mu_f \models \hat{\sigma}_f \text{ and } (\mu_P \cdot \mu_f) \text{ is defined}$
then $\theta, s, (\mu_P \cdot \mu_f) \models \hat{\sigma}$

PROPERTY 3 (OX SOUNDNESS: PRODUCE). Let $\text{lv}(P) \subseteq \text{dom}(\hat{\theta})$.

If $\theta, s, \mu_P \models \hat{\theta}'(P) \text{ and } \theta, s, \mu \models \hat{\sigma} \text{ and } (\mu_P \cdot \mu_f) \text{ is defined}$
then $\exists \theta, \hat{\sigma}'. \theta'|_{\text{lv}(\hat{\sigma})} = \theta \text{ and } \text{produce}(P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow \hat{\sigma}' \text{ and } \theta, s, (\mu_P \cdot \mu_f) \models \hat{\sigma}'$

PROPERTY 4 (UX SOUNDNESS: PRODUCE).

If $\text{produce}(P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow \hat{\sigma}' \text{ and } \theta, s, \mu \models \hat{\sigma}'$
then $\exists \mu_P, \mu_f. \mu = \mu_P \cdot \mu_f \text{ and } \theta, s, \mu_P \models \hat{\theta}(P) \text{ and } \theta, s, \mu_f \models \hat{\sigma}$

Our Consume and Produce Interface

PROPERTY 1 (OX SOUNDNESS: CONSUME).

If $m \in \{OX, EX\}$ and
 $(\forall \hat{\theta}', \hat{\sigma}', o. \text{consume}(m, P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow (o, \hat{\theta}', \hat{\sigma}') \text{ and } \text{SAT}(\hat{\sigma}') \Rightarrow o \neq \text{abort})$
 and $\theta, s, \mu \models \hat{\sigma}$
 then $\exists \mu_P, \mu_f, \hat{\theta}', \hat{\sigma}_f. \text{consume}(m, P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow (\text{ok}, \hat{\theta}', \hat{\sigma}_f)$ and $\mu = \mu_P \cdot \mu_f$
 and $\theta, s, \mu_P \models \hat{\theta}'(P)$ and $\theta, s, \mu_f \models \hat{\sigma}_f$

Result

PROPERTY 2 (UX SOUNDNESS: CONSUME). Gillian's consume and produce operations

If $\text{consume}(P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow (o, \hat{\theta}', \hat{\sigma}')$ satisfy these properties.
 and $\theta, s, \mu_P \models \hat{\theta}'(P)$ and $\theta, s, \mu_f \models \hat{\sigma}'$ and $(\mu_P \cdot \mu_f)$ is defined
 then $\theta, s, (\mu_P \cdot \mu_f) \models \hat{\sigma}$

PROPERTY 3 (OX SOUNDNESS: PRODUCE). Let $\text{lv}(P) \subseteq \text{dom}(\hat{\theta})$.

If $\theta, s, \mu_P \models \hat{\theta}'(P)$ and $\theta, s, \mu \models \hat{\sigma}$ and $(\mu_P \cdot \mu_f)$ is defined
 then $\exists \theta, \hat{\sigma}'. \theta'|_{\text{lv}(\hat{\sigma})} = \theta$ and $\text{produce}(P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow \hat{\sigma}'$ and $\theta, s, (\mu_P \cdot \mu_f) \models \hat{\sigma}'$

PROPERTY 4 (UX SOUNDNESS: PRODUCE).

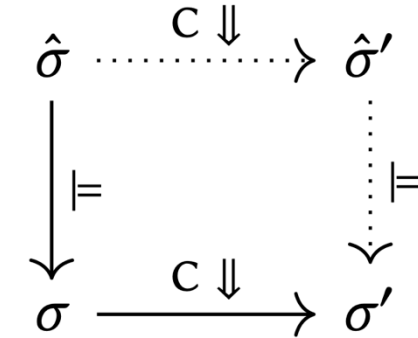
If $\text{produce}(P, \hat{\theta}, \hat{\sigma}) \rightsquigarrow \hat{\sigma}'$ and $\theta, s, \mu \models \hat{\sigma}'$
 then $\exists \mu_P, \mu_f. \mu = \mu_P \cdot \mu_f$ and $\theta, s, \mu_P \models \hat{\theta}(P)$ and $\theta, s, \mu_f \models \hat{\sigma}$

General Soundness Definition

OX soundness

All *concrete* executions can be simulated by a corresponding *symbolic* execution

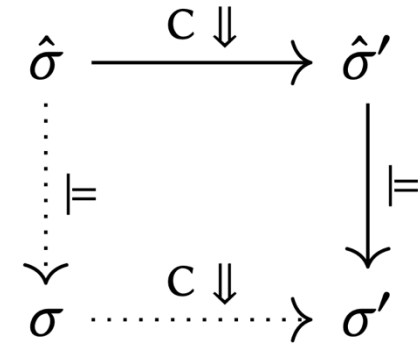
$$\sigma, C \Downarrow \sigma' \wedge \sigma \models \hat{\sigma} \Rightarrow \exists \hat{\sigma}', \hat{\sigma}, C \Downarrow \hat{\sigma}' \wedge \sigma' \models \hat{\sigma}'.$$



UX soundness

All *symbolic* executions can be simulated by a corresponding *concrete* execution

$$\hat{\sigma}, C \Downarrow \hat{\sigma}' \wedge \sigma' \models \hat{\sigma}' \Rightarrow \exists \sigma, \sigma, C \Downarrow \sigma' \wedge \sigma \models \hat{\sigma}.$$



General OX Soundness Result

Concrete memory actions satisfy OX frame property

If $(\mu \cdot \mu_f). \alpha(\vec{v}) \rightsquigarrow o : (\mu', \vec{v}')$
 then $\exists \mu'', o', \vec{v}'' . \mu. \alpha(\vec{v}) \rightsquigarrow o' : (\mu'', \vec{v}'')$ and
 $(o' \neq \text{miss} \Rightarrow (\mu' = \mu'' \cdot \mu_f \text{ and } o' = o \text{ and } \vec{v}'' = \vec{v}'))$

+

Symbolic memory action OX soundness

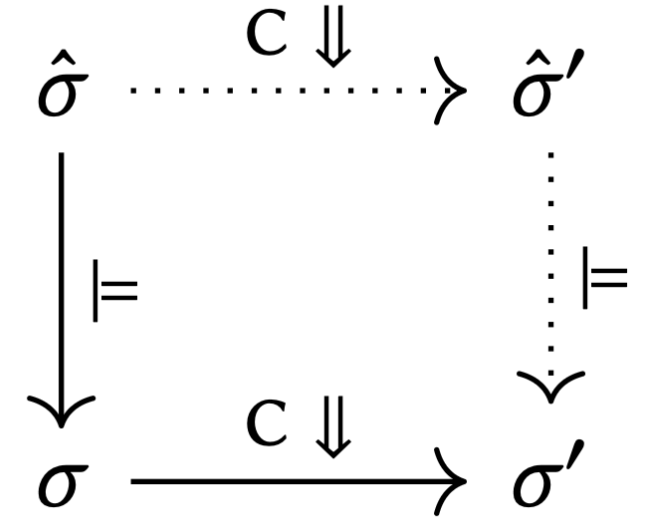
If $\mu. \alpha(\vec{v}) \rightsquigarrow o : (\mu', \vec{v}')$ and $\theta, \mu \models_{\text{Mem}} \hat{\mu}$ and $\llbracket \vec{E} \rrbracket_{\theta} = \vec{v}$ and
 $(\forall o, \hat{\mu}', \hat{\pi}', \vec{E}' . \hat{\mu}. \alpha(\vec{E}) \rightsquigarrow o : (\hat{\mu}', \hat{\pi}', \vec{E}') \text{ and } \llbracket \hat{\pi}' \rrbracket_{\theta} = \text{true and } \llbracket \vec{E}' \rrbracket_{\theta} \neq \perp \Rightarrow o \neq \text{abort})$
 then $\exists \hat{\mu}', \hat{\pi}', \vec{E}', \theta' . \hat{\mu}. \alpha(\vec{E}) \rightsquigarrow o : (\hat{\mu}', \hat{\pi}', \vec{E}')$ and $\theta'|_{\text{lv}(\hat{\mu})} = \theta$
 and $\llbracket \hat{\pi}' \rrbracket_{\theta'} = \text{true and } \theta', \mu' \models_{\text{Mem}} \hat{\mu}' \text{ and } \llbracket \vec{E}' \rrbracket_{\theta'} = \vec{v}'$

+

consume_{Res} & produce_{Res} OX soundness

implies

OX soundness for symbolic execution **with function calls and fold/unfold**



Result

WiSL symbolic execution is sound

Memory-model instances: CSE and Gillian

Memory model name	§	OX	UX	Rocq kLoC	Gillian
Linear model	6.1	✓	✓	1	✗
Linear model with unique-match branching	6.1	✓	✓	1	✗
Linear model with angelic branching	6.1	✗	✓	≈ 0.5	✗
Linear model without negative information	6.1	✓	✗	≈ 0.5	✗
Fractional ownership model	6.2	✓	✓	2	✓*
VeriFast-inspired model for C	6.3	✓	✗	≈ 0.5	✗
Block-offset model for C	6.4	✓	✓	4	✓
Models for OOP languages (e.g., JavaScript)	6.5	✓	✓	✗	✓
CHERI-assembly model	6.6	✓	✗	19	✗
Block-of-trees model for C	6.7	✓	✓	✓*	✓

Yellow checkmark = not applicable, red checkmark = future work, * = variant implemented in Gillian

WiSL demo

Gillian-While with block-offset memory

https://youtu.be/3lLeZB_91L4



Diego Cupello

C demo

*Gillian-C with heap tree memory
(see OPLSS26 fourth lecture)*

<https://youtu.be/zsJLskHjmG0>



Nat Karmios

Published work, ECOOP'24

- **Unified Compositional** Symbolic Execution



- Haskell Implementation



- Gillian Platform

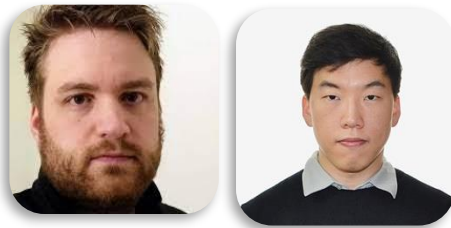


Published Work, OOPSLA'25

- Unified **Parametric** Compositional Symbolic Execution



- **Coq Formalism and Verification**



- Gillian Platform



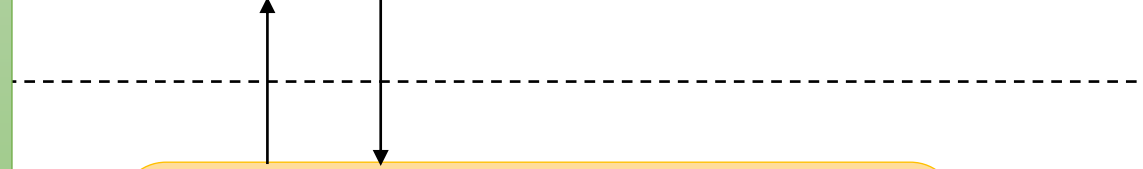
Compositional Symbolic Execution: Foundations

- Symbolic Memory
- Symbolic Actions
- $\text{consume}_{\text{res}}$, $\text{produce}_{\text{res}}$
- OX/UX frame property
- Memory model soundness

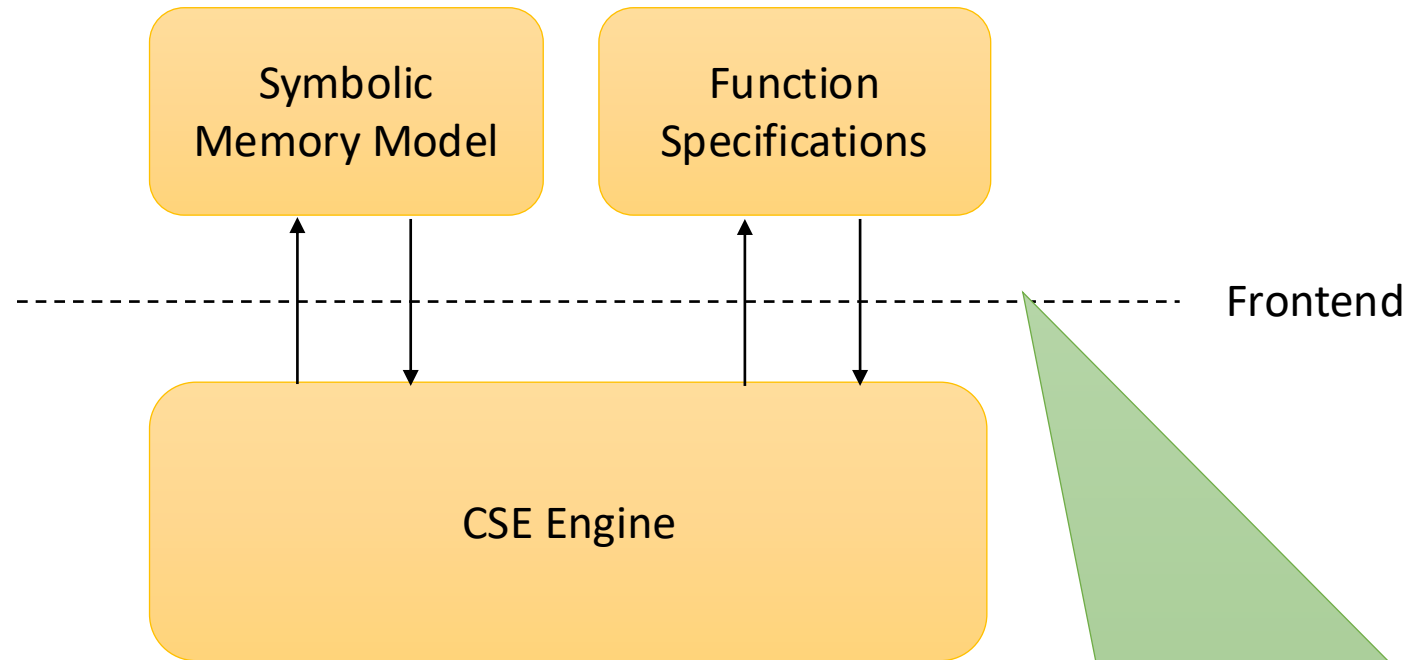
Symbolic
Memory Model

CSE Engine

Frontend



Compositional Symbolic Execution: Foundations



- Function specifications from SL and ISL
- General algebraic interface independent of tool implementation
- General soundness result

CSE Analysis Standardisation....

